Initializing K-Means

The very first step of the K-means algorithm is choosing the initial locations for the cluster centroids ($\mu_1, \ldots, \mu_K$). How you choose these initial values significantly affects the algorithm's performance.

1. Random Initialization Method

The most common and recommended way to initialize the centroids is to pick specific examples from your training set.

- **Constraint:** The number of clusters K must be less than the number of training examples m ($K < m$).
- **The Steps:**
    1. Randomly shuffle the training examples.
    2. Pick the first K examples from the shuffled list.
    3. Set your centroids ($\mu_1, \ldots, \mu_K$) to be equal to the locations of these K examples.

2. The Problem: Local Optima

K-means is sensitive to initialization. Depending on which random points you pick, the algorithm might converge to different solutions.

- **Global Optimum:** The ideal solution where clusters are perfectly separated, resulting in the lowest possible cost (distortion).
- **Local Optima:** Sub-optimal solutions. This happens when K-means gets "stuck." For example, it might split one logical cluster into two while merging two distinct clusters into one.

Because K-means uses gradient-descent-like logic to minimize the cost function J, it can get stuck in a local minimum and fail to find the best clustering.

---

3. The Solution: Multiple Initializations

To solve the problem of local optima, we run the algorithm multiple times with different random starts and pick the best result.

The Algorithm:

1. **Loop** (e.g., 100 times):
    - **Step A:** Randomly initialize the centroids (pick K random examples).
    - **Step B:** Run the K-means algorithm (Assign & Move steps) until it converges.

    - **Step C:** Compute the **Cost Function (Distortion)** J for this specific run.
2. **Select the Best Run:**
    - Compare the cost function J from all 100 runs.
    - Pick the clustering assignment that resulted in the **lowest** cost J.

Typical Usage:

- Performing **50 to 1,000** random initializations is common.
- Doing this virtually guarantees that you find a good solution (the global optimum) rather than a poor local optimum.
- *Note:* Running this more than 1,000 times usually has diminishing returns and becomes computationally expensive.